



BOAZ 미니프로젝트 2 분석 26기

GRAPH-RAG·SELF-RAG 기반 문화유산 지식그래프 대화형 도슨트 시스템

김윤서 성유민 지승훈 김채우





목차

1. 공모전 개요 및 개발 필요성
2. 참고 논문
3. 개발 과정
4. 개발 결과
5. 의의 및 한계점



공모전 개요

제4회 문화체육관광

인공지능·데이터 활용 공모전

데이터로 꿈꾸는 문화, AI로 만나는 새로운 세상



[홈페이지 바로가기]



참가대상

대한민국 국민 누구나 (개인 또는 단체 (5인 이내))
· 단체(기관, 기업도 참여 가능)
· 중복 응모 불가

접수기간

2026. 4. 27.(월) ~ 6. 26.(금), 18:00 까지

공모분야

공모분야	공모부문	공모내용
신기술 활용	ADX 우수사례	AI·디지털 기술을 활용한 문화서비스 우수사례
	ADX 아이디어	AI·디지털 기술을 활용한 문화서비스 사각해소 및 국민체감형 서비스 기획
문화데이터 활용	문화데이터 우수사례	문화데이터를 핵심 자원으로 활용하고 AI 기술도 활용한 제품서비스 우수사례
	문화데이터 아이디어	문화데이터를 핵심 자원으로 활용하고 AI 기술도 활용해 사회문제 해결 또는 서비스 아이디어 기획
데이터분석	-	문화데이터 분석을 통한 정책 현안 해결 또는 분석 인사이트 도출

추진일정



※ 문화데이터 부문 우수사례·아이디어 대상작 <제14회 범정부 공공데이터·AI 활용 창업경진대회> 통합 본선 진출 후보 지정
· 상기 일정은 대내·외 사정에 따라 변경될 수 있음

시상내역

총 상금 5,000만원 (총 15점)

접수방법

홈페이지를 통한 온라인 접수 [https://culture.go.kr/digicon]

문의처

T. 02-6342-8802 E. contest@kcisa.kr

참가분야 및 세부 부문

*우수사례 분야는 제품 및 서비스를 개발

분야	공모 부문	내용
신기술활용	ADX 우수사례	AI·디지털 기술을 활용한 문화서비스 우수사례 ?
	ADX 아이디어	AI·디지털 기술을 활용한 문화서비스 사각해소 및 국민체감형 서비스 기획
문화데이터 활용	문화데이터 우수사례	문화데이터를 핵심 자원으로 활용하고 AI 기술도 활용한 제품서비스 우수사례 ?
	문화데이터 아이디어	문화데이터를 핵심 자원으로 활용하고 AI 기술도 활용해 사회문제 해결 또는 서비스 아이디어 기획
데이터분석	-	문화데이터 분석을 통한 정책 현안 해결 또는 분석 인사이트 도출

신기술 활용 부문

✓역사 왜곡 방지형 문화유산 도슨트 챗봇

문화 유산 지식그래프를 구축한 뒤 SELF RAG를 이용해
근거 있는 양질의 답변을 제공하는 서비스를 구축하고자 함.



개발 필요성

1. 기존 국립중앙박물관 챗봇의 제한적인 질문 처리방식

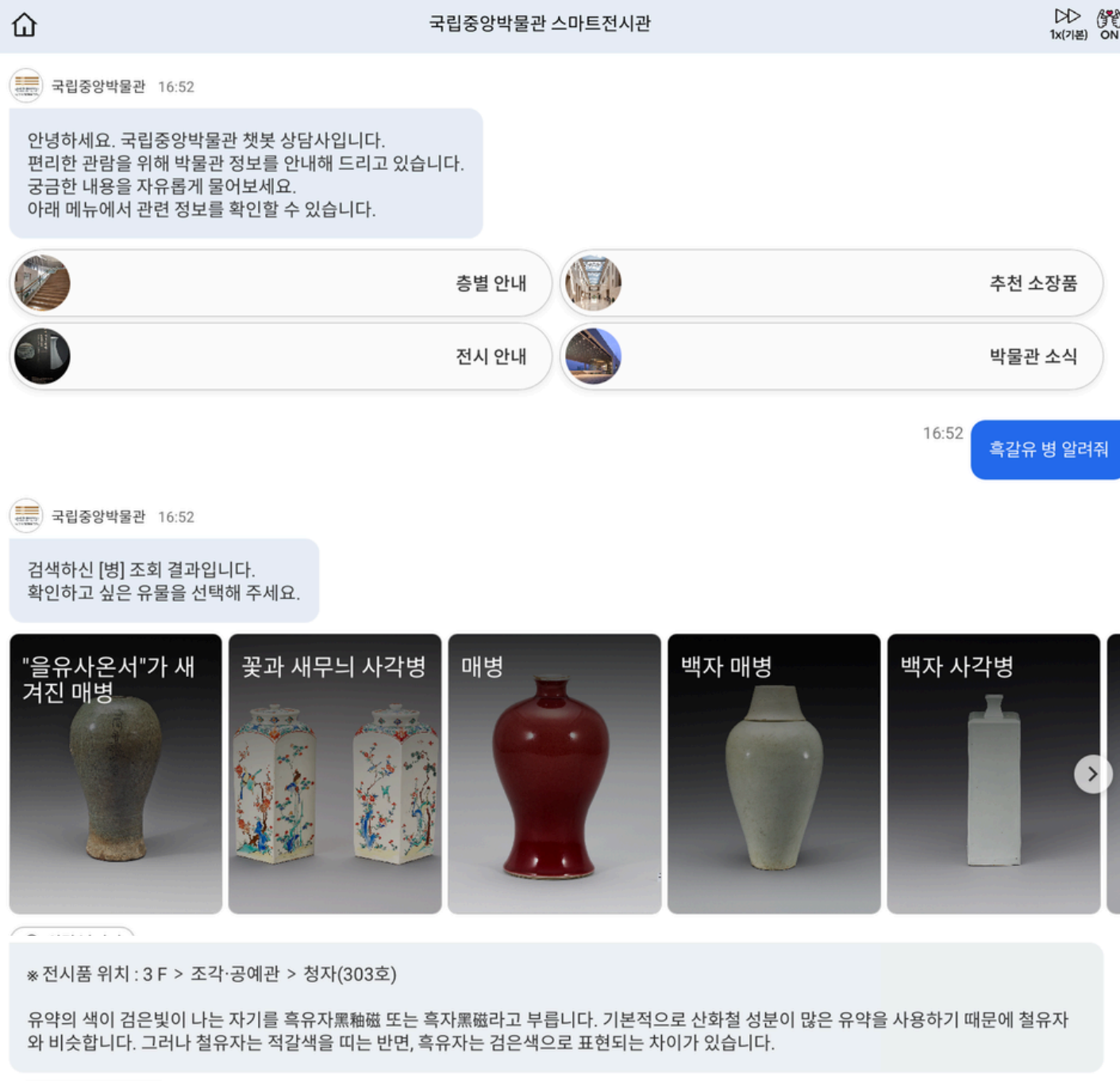
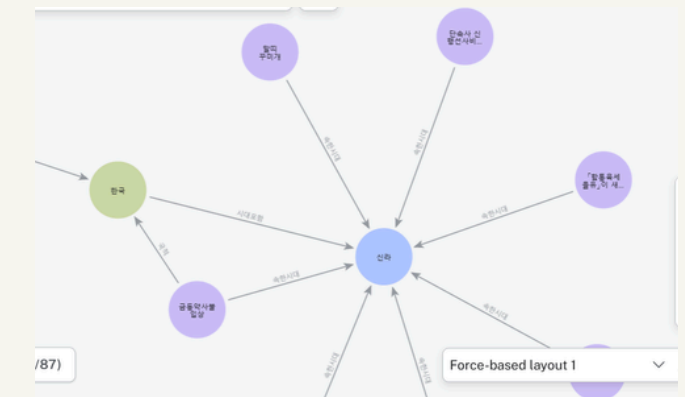
- 정해진 FAQ중심 응답 구조
- 자유로운 자연어 질문 처리의 한계
- 유물 간 관계 기반 탐색 부재
- 멀티턴 대화 기능 부족
- 사용자 수준별 설명 부족
- 단순 정보 나열 형태로 얇은 정보 제공

→ 지식그래프 기반 정보검색 필요

Graph-RAG도입

문화유산 도메인 특화 지식그래프를
구축하여 단순 키워드 검색이 아닌
관계 기반 검색을 가능하게 함.

노드 구성:국적, 분류, 시대, 유물,
작가, 재질, 전시위치





개발 필요성

2. LLM의 유도성 질문 취약 문제

Hallucination

직접 질문:

“석굴암은 언제 만들어졌나요?”
→ 통일신라시대라고 정확히 답변

유도 질문:

“~~고려시대에 만들어진~~ 석굴암의 특징은 무엇인가요?”
→ 실제로는 통일신라시대임에도
잘못된 전제를 수용하여 hallucination 발생

- 시대·유물 정보 등을 왜곡하며 역사적 사실과 다른 정보 생성 가능성
- 역사챗봇으로서 강건성을 갖고 잘못된 전제를 교정해줄 의무 존재

→ **검색 기반 검증방식 필요**

SELF-RAG 도입

RAG

:LLM 입력에 검색을 추가하여 사실 오류를 줄였으나,
k개의 문서를 무분별하게 검색하기 때문에
주제에서 벗어나는 답변이 생성될 수 있음.

Self RAG

:LLM이 언제 검색할지, 검색 결과가 유용한지,
답변의 근거가 있는지 스스로 판단하며
성능은 유지하되 관련성, 근거, 완전성을 확보함.



참고 논문

arXiv:2310.11511v1 [cs.CL] 17 Oct 2023

Preprint.

SELF-RAG: LEARNING TO RETRIEVE, GENERATE, AND CRITIQUE THROUGH SELF-REFLECTION

Akari Asai¹, Zeqiu Wu¹, Yizhong Wang^{1§}, Avirup Sil², Hannaneh Hajishirzi^{1§}

¹University of Washington ²Allen Institute for AI ³IBM Research AI
{akari, zeqiuwu, yizhongw, hannaneh}@cs.washington.edu, avi@us.ibm.com

ABSTRACT

Despite their remarkable capabilities, large language models (LLMs) often produce responses containing factual inaccuracies due to their sole reliance on the parametric knowledge they encapsulate. Retrieval-Augmented Generation (RAG), an ad hoc approach that augments LMs with retrieval of relevant knowledge, decreases such issues. However, indiscriminately retrieving and incorporating a fixed number of retrieved passages, regardless of whether retrieval is necessary, or passages are relevant, diminishes LM versatility or can lead to unhelpful response generation. We introduce a new framework called **Self-Reflective Retrieval-Augmented Generation (SELF-RAG)** that enhances an LM's quality and factuality through retrieval and self-reflection. Our framework trains a single arbitrary LM that adaptively retrieves passages on-demand, and generates and reflects on retrieved passages and its own generations using special tokens, called *reflection* tokens. Generating reflection tokens makes the LM controllable during the inference phase, enabling it to tailor its behavior to diverse task requirements. Experiments show that SELF-RAG (7B and 13B parameters) significantly outperforms state-of-the-art LLMs and retrieval-augmented models on a diverse set of tasks. Specifically, SELF-RAG outperforms ChatGPT and retrieval-augmented Llama2-chat on Open-domain QA, reasoning and fact verification tasks, and it shows significant gains in improving factuality and citation accuracy for long-form generations relative to these models.¹

1 INTRODUCTION

State-of-the-art LLMs continue to struggle with factual errors (Mallen et al., 2023; Min et al., 2023) despite their increased model and data scale (Ouyang et al., 2022). Retrieval-Augmented Generation (RAG) methods (Figure 1 left; Lewis et al. 2020; Guu et al. 2020) augment the input of LLMs with relevant retrieved passages, reducing factual errors in knowledge-intensive tasks (Ram et al., 2023; Asai et al., 2023a). However, these methods may hinder the versatility of LLMs or introduce unnecessary or off-topic passages that lead to low-quality generations (Shi et al., 2023) since they retrieve passages indiscriminately regardless of whether the factual grounding is helpful. Moreover, the output is not guaranteed to be consistent with retrieved relevant passages (Gao et al., 2023) since the models are not explicitly trained to leverage and follow facts from provided passages. This work introduces **Self-Reflective Retrieval-augmented Generation (SELF-RAG)** to improve an LLM's generation quality, including its factual accuracy without hurting its versatility, via on-demand retrieval and self-reflection. We train an arbitrary LM in an end-to-end manner to learn to reflect on its own generation process given a task input by generating both task output and intermittent special tokens (i.e., *reflection tokens*). Reflection tokens are categorized into *retrieval* and *critique* tokens to indicate the need for retrieval and its generation quality respectively (Figure 1 right). In particular, given an input prompt and preceding generations, SELF-RAG first determines if augmenting the continued generation with retrieved passages would be helpful. If so, it outputs a **retrieval** token that calls a retriever model on demand (Step 1). Subsequently, SELF-RAG concurrently processes multiple retrieved passages, evaluating their relevance and then **generating** corresponding task outputs (Step 2). It then generates critique tokens to **criticize** its own output and choose best one (Step 3) in terms of factuality and overall quality. This process differs from conventional RAG (Figure 1 left), which

¹Our code and trained models are available at <https://selfrag.github.io/>.

Main Idea: Reflection tokens – 강화학습의 보상체계

[Retrieve]

외부 문서 검색이 필요한지 판단

[IsRel]

검색된 문서가 질문과 관련 있는지 평가

[IsSup]

생성 답변이 검색 근거로 뒷받침되는지 검증

[IsUse]

최종 답변이 사용자 질문에 유용한지 평가

1. Retrieve on demand

: 필요에 따라 검색 모델을 호출하는 검색토큰 출력

2. Generate segment in parallel

: 검색된 여러 문장을 동시에 처리하여 관련성 평가

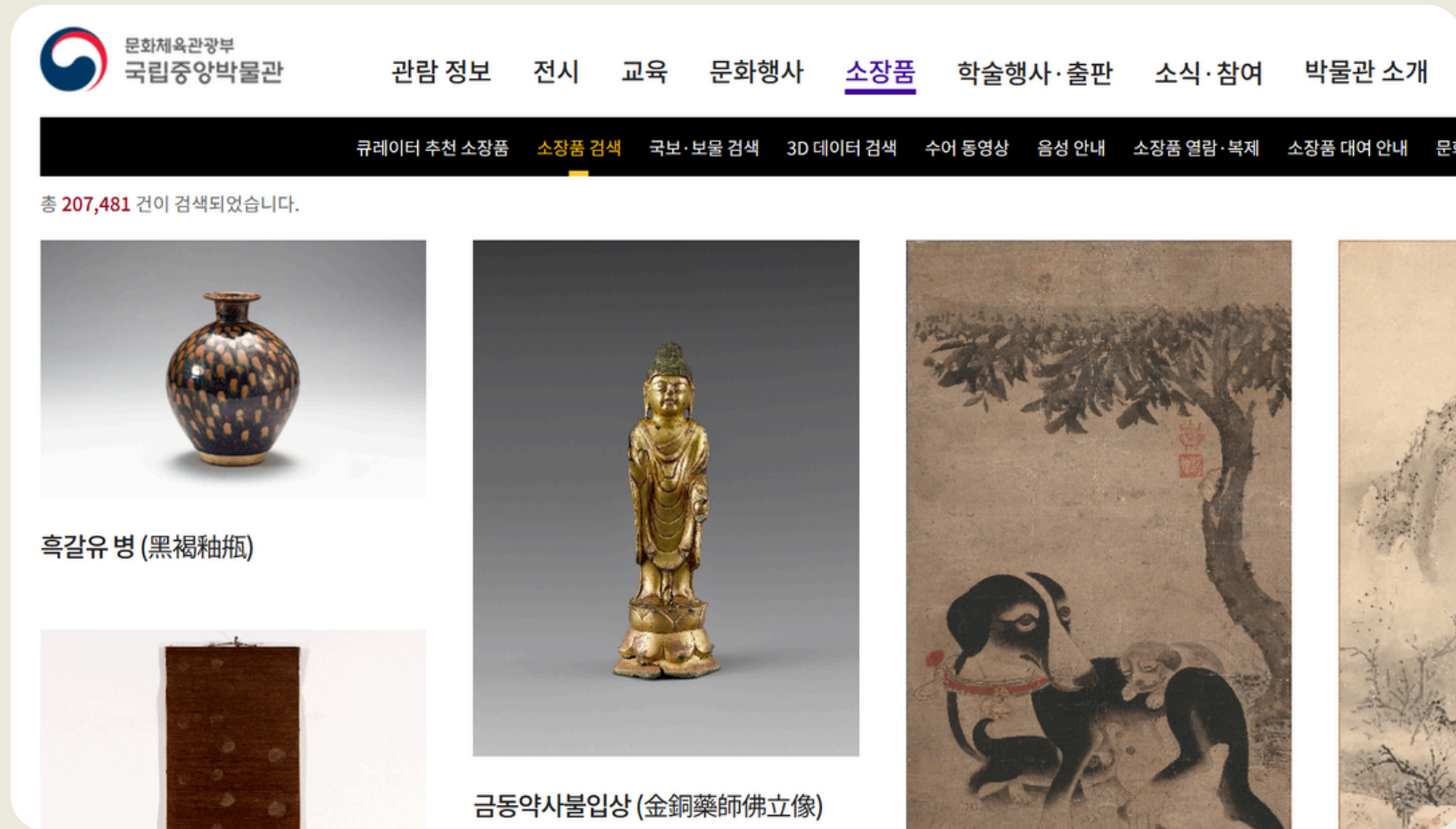
3. Critique outputs and select best segment

: 자체 출력을 검증하여 최적 품질의 출력을 선택



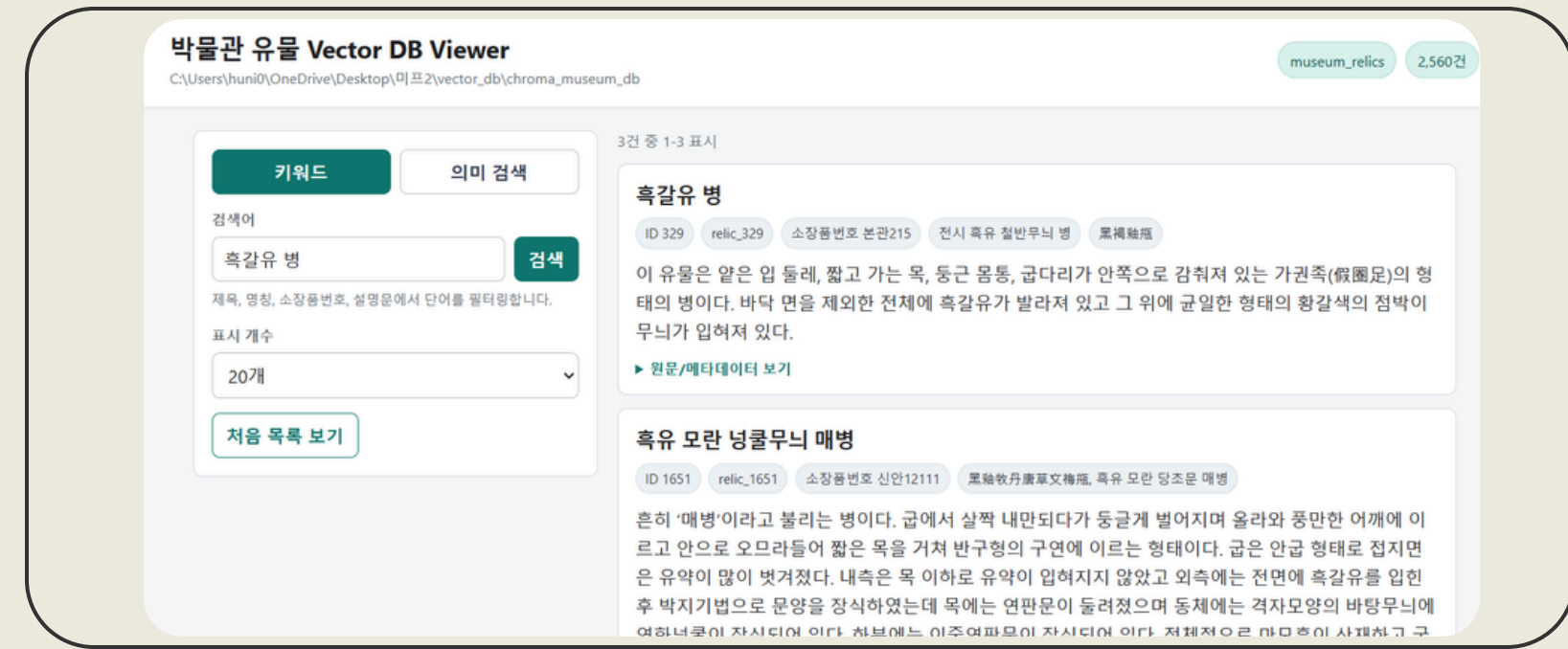
개발 과정

활용 데이터 및 DB 구축

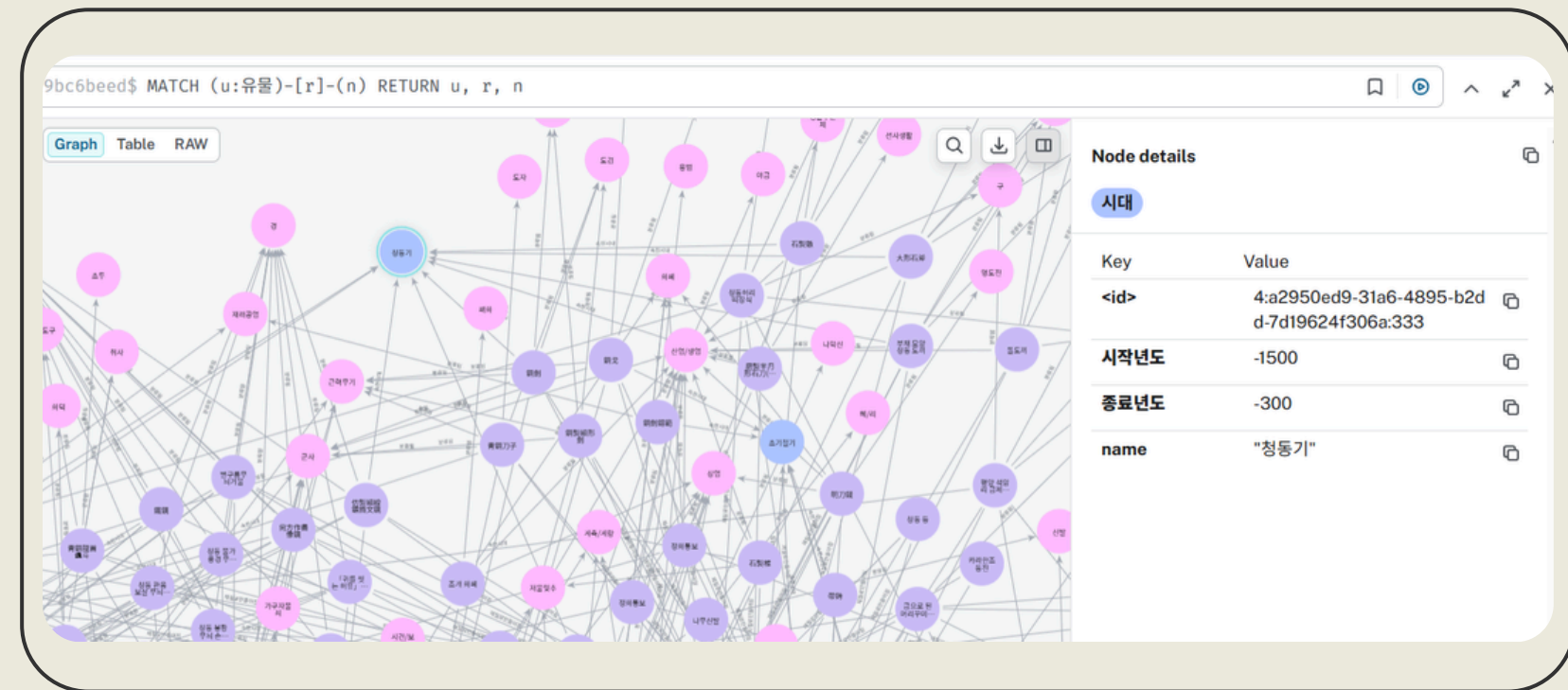


국립 중앙 박물관 사이트의
“소장품 검색”
→ 총 2560개의 유물 정보 크롤링

vector db

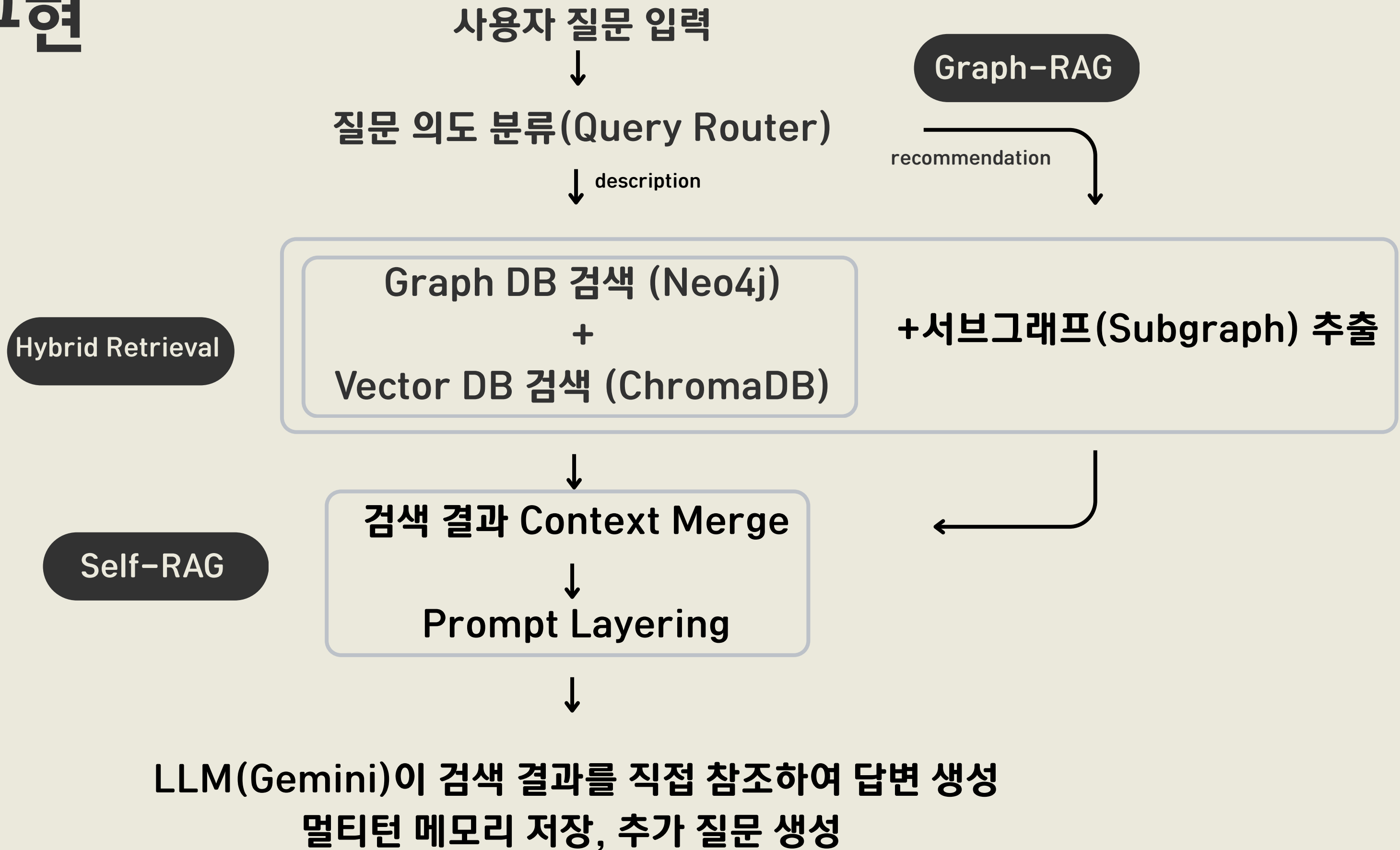


graph db





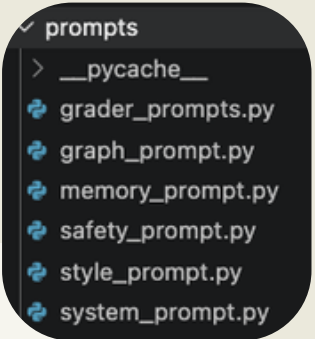
RAG 구현





Prompt Layering

역할(Role), 기억(Context), 추론(Graph Reasoning), 스타일(Persona), 안정성(Self-RAG) 분리 설계



System Prompt

문화유산 AI 도슨트의 기본 역할 정의

- 검색 정보 기반으로만 답변하도록 제한
- 설명 흐름(유물 소개 → 특징 → 역사적 의미) 지정

“사용자에게 친절한 박물관 도슨트처럼 설명하세요.”
“검색 정보에 없는 내용은 추측하지 마세요.”

Memory Prompt

멀티턴 대화를 위한 문맥 유지

- 이전 대화의 유물을 기억하여 흐름 기반 응답
- “그 유물”, “비슷한 거” 같은 지시어 해석 가능

“사용자: “흑갈유 병 알려줘”
사용자: “비슷한 거 더 있어?”
→ 이전 유물(흑갈유 병)을 기준으로 연결

Graph Prompt

Neo4j 관계 기반 유사 유물 추천

- 시대 / 재질 / 전시관 관계 기반 reasoning

“같은 시대의 유물을 추천해줘”
→ 고려시대 노드와 연결된 유물 탐색
“비슷한 재질의 유물 보여줘”
→ 흑유 / 도자기 관계 기반 탐색

Grader Prompt

검색 결과 및 생성 답변 품질 평가

- 검색 결과 relevance 평가
- 답변 grounding 검증
- 질문 재작성(Query Rewrite) 수행

“이 문서가 질문과 관련 있는가?”
→ yes / no 평가
“답변이 검색 정보 기반인가?”
→ hallucination 검사

Style Prompt

사용자 수준에 맞춘 설명 난이도 조정

- 어린이 / 일반인 / 전문가 모드
- 도슨트 말투 유지

어린이 모드:
“옛날 사람들이 사용하던 특별한 병이에요 😊”
전문가 모드:
“길주요 계열의 흑유 자기 양식을 보여줍니다.”

Safety Prompt

환각(Hallucination) 방지 및 Self-RAG 검증

- 검색 정보 기반 여부 확인
- 근거 없는 생성 차단, 답변 품질 검증

“검색 정보에 없는 내용은 답변하지 않습니다.”
“생성 답변이 검색 결과와 일치하는지 재검증”



챗봇 설계

Hybrid Retrieval

Graph Retrieval

Cypher Query Language로 작성
Neo4j의 KG에서 쿼리 수행

```
query = """
MATCH (u:유물)-[r]-(n)

WHERE
  u.title CONTAINS $keyword
OR u.전시명칭 CONTAINS $keyword
OR u.다른명칭 CONTAINS $keyword
OR u.소장품번호 CONTAINS $keyword

RETURN u, r, n
LIMIT 20
"""
```

Graph_retriever.py

```
graph_result = (
  graph_retriever.retrieve_subgraph(
    artifact_name
  )
)
```

서브그래프 저장

Vector Retrieval

```
class VectorRetriever:

    def __init__(self):

        client = chromadb.PersistentClient(
            path=DB_DIR
        )

        self.collection = client.get_collection(
            name=COLLECTION_NAME,
            embedding_function=embedding_function
        )

    def search(self, query):

        results = self.collection.query(
            query_texts=[query],
            n_results=3
        )

        return results
```

Vector_retriever.py

```
vector_result = (
  vector_retriever.search(
    artifact_name
  )
)
```

설명문 저장

LLM Modeling

```
class LLMGenerator:

    def generate(
        self,
        question,
        context,
        chat_history
    ):

        history_text = ""

        for chat in chat_history:

            history_text += (
                f"{chat['role']}: "
                f"{chat['content']}\n"
            )

        full_system_instruction = f"""
{SYSTEM_PROMPT}

{SAFETY_PROMPT}

{MEMORY_PROMPT}

{GRAPH_PROMPT}

{STYLE_PROMPT}
""".strip()

        user_content = f"""
아래 검색 정보를 기반으로만 답변하세요.
```

```
[현재 질문]
{question}
""".strip()

        # =====
        # GEMINI
        # =====

        try:

            response = (
                client.models.generate_content(
                    model="gemini-2.5-flash",

                    contents=user_content,

                    config=types.GenerateContentConfig(
                        system_instruction=
                        full_system_instruction
                    )
                )
            )

            return response.text
```

llm_generator.py

gemini 호출.
프롬프트, 질문, context,
chat history 제공

final_context로 통합



Self-RAG

```
#
# 5. LangGraph 상태 및 노드 제어
#
class GraphState(TypedDict):
    question: str
    persona_guide: str
    generation: str
    documents: List[Document]

def retrieve(state):
    print("==== [1. RETRIEVE (Graph DB 검색)] ====")
    documents = []
    with HybridSearcher() as searcher:
        results = searcher.search(state["question"], n_results=3)
        for r in results:
            documents.append(Document(page_content=r["context"], metadata={"title": r["title"]}))
    return {"documents": documents}

def grade_documents(state):
    print("==== [2. GRADE DOCUMENTS (관련성 평가)] ====")
    filtered_docs = []
    for d in state["documents"]:
        score = retrieval_grader.invoke({"question": state["question"], "document": d.page_content})
        if score.binary_score.lower() == "yes":
            filtered_docs.append(d)
    return {"documents": filtered_docs}

def decide_to_generate(state):
    print("==== [3. ASSESS GRADED DOCUMENTS (분기 결정)] ====")
    if not state["documents"]:
        return "transform_query"
    return "generate"
```

```
def transform_query(state):
    print("==== [TRANSFORM QUERY (질문 재작성)] ====")
    better_question = question_rewriter.invoke({"question": state["question"]})
    return {"question": better_question}

def generate(state):
    print("==== [4. GENERATE (Prompt Layering 기반 답변 생성)] ====")
    context_text = "\n\n".join([d.page_content for d in state["documents"]])
    persona = state.get("persona_guide", "")

    generation = rag_chain.invoke({
        "context": context_text,
        "question": state["question"],
        "persona_guide": persona
    })
    return {"generation": generation}

def grade_generation_v_documents_and_question(state):
    print("==== [5. CHECK HALLUCINATIONS (환각 및 정답 평가)] ====")
    documents = "\n\n".join([d.page_content for d in state["documents"]])

    score = groundedness_grader.invoke({"documents": documents, "generation": state["generation"]})
    if score.binary_score.lower() == "yes":
        score = answer_grader.invoke({"question": state["question"], "generation": state["generation"]})
        if score.binary_score.lower() == "yes":
            return "relevant"
        return "not relevant"
    return "hallucination"
```

1. Retrieve: Hybrid Search 검색 결과를 LangChain Document 형태로 변환

2. Grade Documents: 검색된 문서가 질문과 실제로 관련 있는지 LLM 기반 grader로 평가하여 관련성이 낮은 문서 제거

3. Transform Query: 검색 결과가 부족할 경우 질문을 재작성하여 검색

4. Generate: Prompt Layering 기반 답변 생성

5. Hallucination Check: 생성 답변이 실제 검색 정보 기반인지 추가 평가



```
# — 메인 영역 —
st.markdown("""
<div class="main-title">
    <h1>🏛️ 국립중앙박물관 AI 도슨트</h1>
    <p>유물에 대해 무엇이든 물어보세요</p>
</div>
<hr class="divider">
""", unsafe_allow_html=True)

# 유물 카드 렌더링 함수
def render_artifact_card(artifact: dict, rank: int):
    graph = artifact.get("graph", {})
    tags_era = "".join([f'<span class="artifact-tag era">{v}</span>' for v in graph.get("시대", [])])
    tags_mat = "".join([f'<span class="artifact-tag mat">{v}</span>' for v in graph.get("재질", [])[:3]])
    tags_loc = "".join([f'<span class="artifact-tag loc">{v}</span>' for v in graph.get("전시위치", [])])
    tags_cat = "".join([f'<span class="artifact-tag">{v}</span>' for v in graph.get("분류", [])[:2]])

    st.markdown(f"""
<div class="artifact-card">
    <h4>#{rank} {artifact.get('title', '')}
        <span style="font-size:12px; color:#999; font-weight:400; margin-left:8px;">
            {artifact.get('소장품번호', '')}
        </span>
    </h4>
    {tags_era}{tags_mat}{tags_loc}{tags_cat}
</div>
""", unsafe_allow_html=True)
```

```
# 메시지 없을 때 환영 화면
if not st.session_state.messages:
    st.markdown("""
<div style="text-align:center; padding: 60px 20px; color: #9a8a70;">
    <div style="font-size: 56px; margin-bottom: 16px;">🏛️</div>
    <div style="font-size: 18px; font-weight: 500; color: #3a3020; margin-bottom: 8px;">
        안녕하세요! 국립중앙박물관 AI 도슨트입니다.
    </div>
    <div style="font-size: 14px; line-height: 1.8;">
        유물의 역사, 시대적 배경, 제작 방식 등<br>
        궁금한 것을 자유롭게 질문해보세요.<br><br>
    </div>
</div>
""", unsafe_allow_html=True)

# 채팅 히스토리 출력
for msg in st.session_state.messages:
    with st.chat_message(msg["role"], avatar="🏛️" if msg["role"] == "assistant" else "👤"):
        st.markdown(msg["content"])
        if msg["role"] == "assistant" and "sources" in msg:
            with st.expander(f"📖 참고 유물 {len(msg['sources'])}건", expanded=False):
                for art in msg["sources"]:
                    render_artifact_card(art, art.get("rank", 1))
```

사용자와 자연스럽게 상호작용하는 AI 도슨트 경험 제공을 목표로 UI/UX 설계.

1. 정해진 명령어 없이 자유로운 질문이 가능한 자연어 기반 대화형 인터페이스
2. 이전 대화 맥락을 기억하고, 추가 질문을 유도하며 정보 확장 제공
3. 페르소나 기반 설명 스타일 설정으로 사용자 수준에 따라 난이도 조절 가능



파일 구조

└─ 📁 llm #LLM 및 프롬프트 관리
| └─ 📄 llm_generator.py #핵심 프롬프트 계층화 및 답변 생성

└─ 📁 router #의도 파악 모듈
| └─ 📄 query_router.py #질문 분석 후 검색

└─ 📁 prompts #역할 별 프롬프트 관리
| └─ 📄 grader_prompts.py #문서/환각 평가 프롬프트
| └─ 📄 graph_prompt.py #유물 관계 기반 추론 프롬프트
| └─ 📄 memory_prompt.py #이전 대화 맥락 참고 프롬프트
| └─ 📄 safety_prompt.py #환각 방지 프롬프트
| └─ 📄 style_prompt.py #페르소나별 맞춤 제공 프롬프트
| └─ 📄 system_prompt.py #기본 시스템 프롬프트

└─ 📄 hybrid_search.py #Graph DB+Vector DB 통합 검색
└─ 📄 api_graph_rag.py #Graph RAG API 서버
└─ 📄 api_self_rag.py #Graph RAG+SELF RAG API 서버
└─ 📄 chatbot_app.py #Streamlit 기반 챗봇 인터페이스
└─ 📄 museum_collections.json #국립중앙박물관 유물 보관 데이터베이스
└─ 📄 .env #env 파일

└─ 📁 graph #지식 그래프 모듈
| └─ 📄 neo4j_client.py #DB 연결
| └─ 📄 graph_retriever.py #유물명 기반 서브그래프 검색



AI 도슨트

국립중앙박물관 유물 안내 서비스

관람객 유형

- ☐  어린이
- ☒  초보
- ☐  역사 전문가

친절하게 기본 설명

DB 현황

항목	수량
유물	2,560개
시대	72종
재질	60종
분류	373종

[Deploy](#)

국립중앙박물관 AI 도슨트

유물에 대해 무엇이든 물어보세요



안녕하세요! 국립중앙박물관 AI 도슨트입니다.

유물의 역사, 시대적 배경, 제작 방식 등
궁금한 것을 자유롭게 질문해보세요.

안녕하세요 조선시대 I





SELF-RAG 최적화

문제 발견

- 매 질문마다 LLM을 호출하여 API 비용 발생
- 응답 시간이 지나치게 길어짐

쿼리 캐싱

- 같은 질문이 반복되면 검색 및 LLM 파이프라인 전면 생략
- 캐시 메모리에서 즉시 반환

계층적 판단

- 모든 문서를 LLM에게 맡기지 않고 python 기반 1차 필터 도입
- 검색된 문서에 핵심 키워드가 없으면 LLM 호출 없이 drop 처리



SELF-RAG 최적화

첫 질문 시 캐시에 존재하지 않는 경우 검색 시작

[새 요청 수신] 청자 향로에 대해 설명해줘.

캐시에 없음

[검색 시작] 질문: 청자 향로에 대해 설명해줘.

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
Loading weights: 100%| 199/19

[계층적 판단 시작] 추출된 키워드: ['청자', '향로에', '대해', '설명해줘.']

[1차 필터 통과] LLM 으로 넘김: 청자 양각 도철 무늬 사각 향로

[2차 LLM 합격]: 청자 양각 도철 무늬 사각 향로

[1차 필터 통과] LLM 으로 넘김: 청자 향로

[2차 LLM 합격]: 청자 향로

[1차 필터 통과] LLM 으로 넘김: 청자 팔괘무늬 향로

[2차 LLM 합격]: 청자 팔괘무늬 향로

[답변 생성 중...]

쿼리 캐시에 저장 완료

같은 질문 재요청 시, 0.0004초 안에 캐시에서 가져옴

[새 요청 수신] 청자 향로에 대해 설명해줘

캐시에서 즉시 반환 (소요 시간: 0.0004초)

[새 요청 수신] 반가사유상에 대해 설명해줘

캐시에 없음

[검색 시작] 질문: 반가사유상에 대해 설명해줘

[계층적 판단 시작] 추출된 키워드: ['반가사유상에', '대해', '설명해줘']

[1차 필터 탈락] 키워드 매칭 실패 (LLM 생략): 금동반가사유상

[1차 필터 통과] LLM 으로 넘김: 금동 반가사유상

[2차 LLM 합격]: 금동 반가사유상

[1차 필터 탈락] 키워드 매칭 실패 (LLM 생략): 비암사 비상

[답변 생성 중...]

쿼리 캐시에 저장 완료

“반가사유상에 대해 설명해줘” 질문 시, 관련 없는
“비암사 비상” 문서는 1차 필터에서 탈락하여 연산량 절감

(“반가사유상” 대신 “반가사유상에” 매칭하는 단어를 확인함
한국어 형태소 분석기를 도입해 명사만 추출하도록 고도화할 계획)



성능 평가

평가지표



ragas

RAG 파이프라인을 평가하고
성능을 정량화하는 프레임워크

+ 시대 오류, 출처 위조 등 도메인
맞춤형 데이터셋 증강 후 평가

1. Context Recall 📌 LLMContextRecall()

정답을 말하기 위해 필요한 핵심 유물 정보(Context)를
DB에서 잘 검색했는가?

2. Faithfulness 📌 Faithfulness()

LLM이 만들어낸 답변이 검색한 유물 문서의 내용에만
근거하고 있는가? (환각 현상이 없는지 평가)

3. Factual Correctness 📌 FactualCorrectness()

최종 생성된 답변이 미리 작성해둔 정답과 역사적 사실
측면에서 일치하는가?



성능 평가

자체 평가 데이터셋 구축

Question: "통일신라 시대 청자를 알려줘" (함정 질문)

Context: "이것은 고려시대 청자입니다."

Answer: "통일신라 시대의 청자는..."

Ground Truth: "해당 유물은 통일신라가 아닌 고려시대 유물입니다."

→ 4가지 요소를 묶어 LLM이 객관적 점수를 매기도록 자동화

평가 결과

- Faithfulness(신뢰성): 0.8645
- Context Recall(문서 재현율): 0.5648
(환각 제어를 최우선으로 두어 관련 없는 문서는 강하게 필터링)
- Factual Correctness(사실 정확성): 0.3111
(정량적 평가 프레임워크 구조의 한계)



의의 & 한계점

의의

- 복합 검색을 RAG 파이프라인에 새롭게 도입
- 단순 검색->생성을 넘어 AI가 스스로 검색 문서 적합성을 평가하는 자체 검증 구조 도입
- 챗봇의 대답이 실제 정보에 근거했는지 스스로 재평가하여 역사적 신뢰성 확보
- 답변 생성 전에 계층적 문서 필터링으로 API 비용 절약 및 노이즈 최소화
- 모호한 질문도 스스로 판단하여 의미와 관계에 맞는 능동적 쿼리 재작성으로 검색 한계 극복

한계점

- 다중 연쇄 호출로 인한 응답 속도 지연 발생
- Vector와 Graph DB를 동시 운영함에 따라 신규 데이터 추가 시 파이프라인 복잡도 증가
- 프롬프팅을 너무 강하게 설정할 경우, 오개념 정정에 필요한 단서까지 기각시키는 문제 발생
- 띄어쓰기 및 단순 키워드 중심 매칭의 한계
- 풍부한 지식망 구축을 위해 대규모 메타데이터 확충 필요



감사합니다

Q&A

